



ZkMinterTrigger Review

November 5, 2025

Prepared for ZKsync

Conducted by:

Richie Humphrey (devtooligan)

About the ZKsync ZkMinterTrigger Review

ZKsync Era is a Layer 2 ZK rollup that uses cryptographic validity proofs to provide scalable and low-cost transactions on Ethereum. The ZkMinterTrigger contract provides a hook mechanism that can automatically execute predefined sequences of calls (targets, calldata, and values) when minting operations occur.

About Offbeat Security

Offbeat Security is a boutique security company providing unique security solutions for complex and novel crypto projects. Our mission is to elevate the blockchain security landscape through invention and collaboration.

Summary & Scope

The [src](#) folder was reviewed at commit [aa60218](#).

The following **2 files** were in scope:

- src/ZkMinterTriggerV1.sol
- src/ZkMinterTriggerV1Factory.sol

Summary of Findings

Identifier	Title	Severity	Fixed
L-01	Unchecked ETH transfer in token recovery may emit incorrect event	Low	PR#47
I-01	Trigger function can be executed independently and repeatedly	Informational	PR#48
I-02	Immutable recovery address could lead to permanently locked tokens	Informational	Ack. The solution would involve adding a permissioned admin role which carries additional own risks.
I-03	Trigger calls to addresses without code succeed silently	Informational	PR#49
I-04	Trigger allows ETH transfer to zero address due to missing target validation	Informational	PR#50

Additional Recommendations

Note: The `ZkMinterTriggerV1Factory` contract accepts a bytecode hash as a constructor parameter without verifying it matches the actual deployment bytecode of

`ZkMinterTriggerV1`. This could lead to incorrect address predictions by the `getMinter()` function if the provided hash doesn't match the actual deployed contract bytecode. This issue been reported on previous versions which the project has acknowledged and stated they will take care to deploy the factory with the correct bytecode hash.

Detailed Findings

Low Findings

[L-01] Unchecked ETH transfer in token recovery may emit incorrect event

Description

The `recoverTokens()` function in `ZkMinterTriggerV1` does not check the success of ETH transfers when recovering native tokens. When the `_token` parameter is set to the ETH sentinel address (`0xEeeeeEeeeEeEeeEeEeEeEeEeEeEeEeEeE`), the function uses a low-level `.call()` to transfer ETH but ignores the return value:

```
function recoverTokens(address _token, uint256 _amount) external virtual {
    _checkRole(DEFAULT_ADMIN_ROLE, msg.sender);
    if (_token == address(0xEeeeeEeeeEeEeeEeEeEeEeEeEeEeEeEeE)) {
        RECOVERY_ADDRESS.call{value: _amount}(""); // Return value not checked
    } else {
        IERC20(_token).safeTransfer(RECOVERY_ADDRESS, _amount);
    }
    emit TokensRecovered(msg.sender, _token, _amount, RECOVERY_ADDRESS);
}
```

If the ETH transfer fails (for example, if `RECOVERY_ADDRESS` is a contract with a reverting fallback function), the function will continue execution and emit the `TokensRecovered` event incorrectly, suggesting that tokens were successfully recovered when they were not.

This issue does not affect ERC20 token recovery since `safeTransfer()` properly checks for success and reverts on failure.

Recommendation

Check the success of the ETH transfer and revert if it fails:

```
function recoverTokens(address _token, uint256 _amount) external virtual {
    _checkRole(DEFAULT_ADMIN_ROLE, msg.sender);
    if (_token == address(0xEeeeeEeeeEeEeeEeEeEeEeEeEeEeEeEeE)) {
        - RECOVERY_ADDRESS.call{value: _amount}("");
        + (bool success, ) = RECOVERY_ADDRESS.call{value: _amount}("");
        + if (!success) revert ZkMinterTriggerV1__RecoveryTransferFailed();
    } else {
        IERC20(_token).safeTransfer(RECOVERY_ADDRESS, _amount);
    }
    emit TokensRecovered(msg.sender, _token, _amount, RECOVERY_ADDRESS);
}
```

Informational Findings

[I-01] Trigger function can be executed independently and repeatedly


```
IERC20(_token).safeTransfer(RECOVERY_ADDRESS, _amount);
}
```

Given the variety of tokens and protocols that might interact with this contract through the trigger mechanism, maintaining flexibility in the recovery address would provide better long-term security for recovered assets.

Recommendation

Consider implementing the recovery address as a state variable that can be updated by the `DEFAULT_ADMIN_ROLE`:

```
- address public immutable RECOVERY_ADDRESS;
+ address public recoveryAddress;

+ event RecoveryAddressUpdated(address indexed oldAddress, address indexed newAddress);

constructor(
    IMintable _mintable,
    address _admin,
    address[] memory _targetAddresses,
    bytes[] memory _calldatas,
    uint256[] memory _values,
    address _recoveryAddress
) {
    // ... existing checks ...
- RECOVERY_ADDRESS = _recoveryAddress;
+ recoveryAddress = _recoveryAddress;
}

+ function updateRecoveryAddress(address _newRecoveryAddress) external {
+     _checkRole(DEFAULT_ADMIN_ROLE, msg.sender);
+     if (_newRecoveryAddress == address(0)) {
+         revert ZkMinterTriggerV1__InvalidRecoveryAddress();
+     }
+     address oldAddress = recoveryAddress;
+     recoveryAddress = _newRecoveryAddress;
+     emit RecoveryAddressUpdated(oldAddress, _newRecoveryAddress);
+ }

function recoverTokens(address _token, uint256 _amount) external virtual {
    _checkRole(DEFAULT_ADMIN_ROLE, msg.sender);
    if (_token == address(0xEeeeeEeeeEeEeeEeEeEEEEEEEEEEEEEEEEEEEE)) {
- RECOVERY_ADDRESS.call{value: _amount}("");
+ recoveryAddress.call{value: _amount}("");
    } else {
- IERC20(_token).safeTransfer(RECOVERY_ADDRESS, _amount);
+ IERC20(_token).safeTransfer(recoveryAddress, _amount);
    }
- emit TokensRecovered(msg.sender, _token, _amount, RECOVERY_ADDRESS);
+ emit TokensRecovered(msg.sender, _token, _amount, recoveryAddress);
}
```

[I-03] Trigger calls to addresses without code succeed silently

Summary

The `ZkMinterTriggerV1` contract's `trigger()` function uses low-level calls that will silently succeed when calling addresses with no code deployed. This could lead to unexpected behavior where trigger operations appear successful but no actual state changes occur at the target addresses.

Description

The `trigger()` function in `ZkMinterTriggerV1` executes a series of low-level calls to predefined target addresses:

```
function trigger() public payable virtual {
    _revertIfClosed();
    _requireNotPaused();
    _checkRole(MINTER_ROLE, msg.sender);

    for (uint256 i = 0; i < targets.length; i++) {
        (bool success,) = targets[i].call{value: values[i]}(calldatas[i]);
        if (!success) {
            revert ZkMinterTriggerV1__TriggerCallFailed(i, targets[i]);
        }
    }

    emit TriggerExecuted(msg.sender);
}
```

The issue is that low-level `call()` operations to addresses without deployed code (EOAs or undeclared contract addresses) return `success = true` with empty return data. This means the trigger function will complete successfully even if the intended target contracts don't exist, potentially causing:

1. ETH to be sent to EOAs or undeclared addresses unintentionally
2. Critical operations to be skipped without any error indication
3. Misleading `TriggerExecuted` events suggesting successful execution

This behavior is particularly concerning for scenarios involving deterministic CREATE2 addresses where contracts are intended to be deployed later, or when target contracts might be destroyed via `SELFDESTRUCT`.

Recommendation

Consider adding code length checks in the `trigger()` function before executing calls. This approach is preferable to checking in the factory as it allows for flexibility with deterministic addresses that may be deployed after the trigger contract:

```
function trigger() public payable virtual {
    _revertIfClosed();
```

```

    _requireNotPaused();
    _checkRole(MINTER_ROLE, msg.sender);

    for (uint256 i = 0; i < targets.length; i++) {
+       // Check that the target has code deployed
+       if (targets[i].code.length == 0) {
+           revert ZkMinterTriggerV1__NoCodeAtTarget(i, targets[i]);
+       }
+       (bool success,) = targets[i].call{value: values[i]}(calldatas[i]);
        if (!success) {
            revert ZkMinterTriggerV1__TriggerCallFailed(i, targets[i]);
        }
    }

    emit TriggerExecuted(msg.sender);
}

```

[I-04] Trigger allows ETH transfer to zero address due to missing target validation

Description

The `ZkMinterTriggerV1` contract's constructor does not validate that target addresses are non-zero. When the `trigger()` function is executed, it performs low-level calls to each configured target with the specified ETH value. If a target address is set to `address(0)`, the call will succeed but permanently burn the ETH, as transfers to the zero address are valid in Ethereum but result in irretrievable funds.

The issue occurs because:

1. The constructor accepts the `_targetAddresses` array without validating individual addresses
2. The `targets` array is immutable after deployment, making any misconfiguration permanent
3. Low-level calls to `address(0)` with ETH succeed silently, burning the funds
4. The `trigger()` function continues execution and emits the `TriggerExecuted` event normally

This creates a scenario where ETH can be permanently lost on each `trigger()` execution if a zero address was mistakenly configured during deployment.

Recommendation

Add validation in the constructor to ensure no target addresses are zero:

```

constructor(
    ...

```

```
        if (_targetAddresses.length != _calldatas.length || _calldatas.length != _va  
            revert ZkMinterTriggerV1__ArrayLengthMismatch();  
    }  
  
+   for (uint256 i = 0; i < _targetAddresses.length; i++) {  
+       if (_targetAddresses[i] == address(0)) {  
+           revert ZkMinterTriggerV1__InvalidTarget();  
+       }  
+   }
```